Laboratorio 2 — TF-IDF vs Word2Vec

Alvaro Andres Castiblanco Lopez[1], Bryan Mauricio Guzmán García[2], Juan Carlos Tovar Orjuela[3] y Lorraine Jazlady Rojas Parra[4]

[1]Maestría en Ingeniería de la Información, E-mail: a.castiblancol@uniandes.edu.co.
[2]Maestría en Arquitectura de Tecnologías de Información, E-mail: b.guzmang@uniandes.edu.co.
[3]Maestría en Ingeniería de la Información, E-mail: jc.tovaro1@uniandes.edu.co.
[4]Maestría en Ingeniería de la Información, E-mail: lj.rojasp1@uniandes.edu.co.

1. Introducción

Este informe documenta el desarrollo del **Laboratorio 2** del curso de Sistemas de Recomendación. El objetivo fue estudiar un problema de *filtrado por contenido* sobre el dataset de *LibraryThing*, utilizando dos estrategias de representación de ítems: una representación dispersa basada en **TF-IDF** y una representación densa basada en **Word2Vec**. A lo largo del laboratorio se construyó una matriz documento–término a partir de conceptos de DBpedia [1], se aplicó selección de características mediante chi-cuadrado, se entrenó un clasificador KNN por usuario para predecir preferencias binarias y, finalmente, se comparó el desempeño de TF-IDF frente a Word2Vec en un escenario de recomendación basada en contenido.

1.1. Flujo general del laboratorio

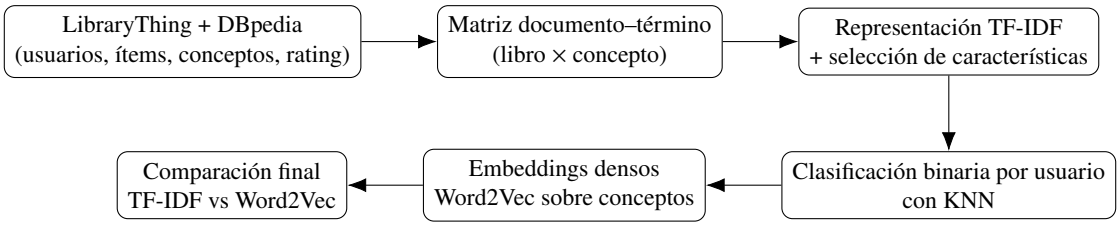

Figure 1: Pipeline seguido en el laboratorio.

2. Carga del dataset y construcción de la matriz documento–término

El dataset utilizado tiene una estructura distinta a la del laboratorio anterior: cada fila representa una interacción entre un usuario y un libro, replicada una vez por cada concepto de DBpedia asociado al libro. En consecuencia, un mismo rating aparece repetido tantas veces como conceptos describen al ítem. Esta organización permite reinterpretar cada libro como un conjunto de conceptos semánticos y construir a partir de allí una matriz documento–término [1].

A partir del dataframe original se construyeron tres tablas auxiliares. La primera, `df_libros`, contiene el identificador y el nombre de cada libro. La segunda, `df_conceptos`, contiene el catálogo de conceptos únicos, identificados por `featureID` y su nombre corto `featureShortname`. La tercera, `df_libros_concepto`, representa la relación libro–concepto sin duplicados, es decir, una fila por cada par único entre un libro y una característica semántica.

Con esta última tabla se generó una matriz binaria libro–concepto mediante una operación de *pivot*. Las filas representan libros, las columnas representan conceptos y cada celda toma valor 1 cuando el libro contiene el concepto y 0 en caso contrario. El resultado fue una matriz de dimensión 6164×12246, lo que confirma que el problema se modela inicialmente en un espacio de muy alta dimensionalidad.

featureID **DBbook_ItemID**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	...	**13992**
1	1	1	1	1	1	1	1	1	1	1	...	0
2	1	1	1	1	1	1	1	0	1	1	...	0
3	0	0	1	1	1	1	1	1	0	0	...	0
5	0	0	1	1	1	1	1	0	0	0	...	0
7	0	0	1	1	1	1	1	0	0	0	...	0
...	...	...	...	...	...	...	...	...	...	...	...	...
8164	0	0	1	1	1	1	1	0	0	0	...	0
8166	0	0	0	0	0	0	0	0	0	0	...	0
8167	0	0	0	0	0	0	0	0	0	0	...	0
8168	0	0	1	1	1	1	1	0	0	0	...	0
8169	0	0	1	1	1	1	1	0	0	0	...	0

Tabla 1: Ejemplo de matriz libro–concepto binaria. Las filas representan `DBbook_ItemID` y las columnas `featureID`. Un valor de 1 indica que el libro contiene el concepto y 0 en caso contrario.

3. Exploración de la representación basada en conceptos

3.1. Número de ítems por característica

Al sumar la matriz libro–concepto por columnas se obtuvo un vector de frecuencias documentales, donde cada valor indica en cuántos libros aparece un concepto. La visualización de esta distribución mostró una fuerte asimetría: la mayoría de las características aparece en muy pocos ítems, mientras que solo un pequeño grupo aparece en miles de libros.

Este comportamiento indica que existen muchas características raras o muy específicas y pocas características generales. Desde la perspectiva del recomendador, esto es importante porque las características demasiado frecuentes aportan poca capacidad de discriminación entre libros, mientras que las poco frecuentes pueden ser más útiles para distinguir ítems, aunque también introducen dispersión y sparsity.

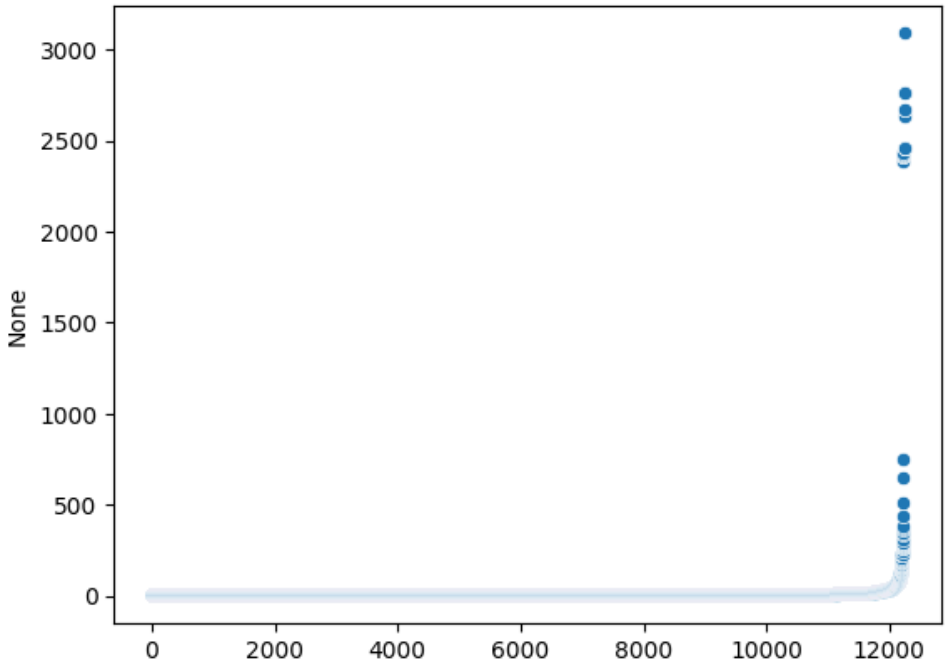

Figure 2: Número de ítems asociados a cada característica.

3.2. Número de características por ítem

Al sumar la matriz por filas se obtuvo el número de características asignadas a cada libro. La visualización correspondiente mostró que la mayoría de los ítems tiene una cantidad intermedia de conceptos, aunque también existen libros con muy pocos atributos y otros con una descripción mucho más extensa.

Esto sugiere que los ítems no están descritos de manera homogénea. Algunos libros cuentan con una representación semántica rica, mientras que otros están relativamente subdescritos. En términos prácticos, los libros con pocas características son más difíciles de comparar de forma robusta, mientras que los libros con una representación más completa ofrecen más señales para medir similitud.

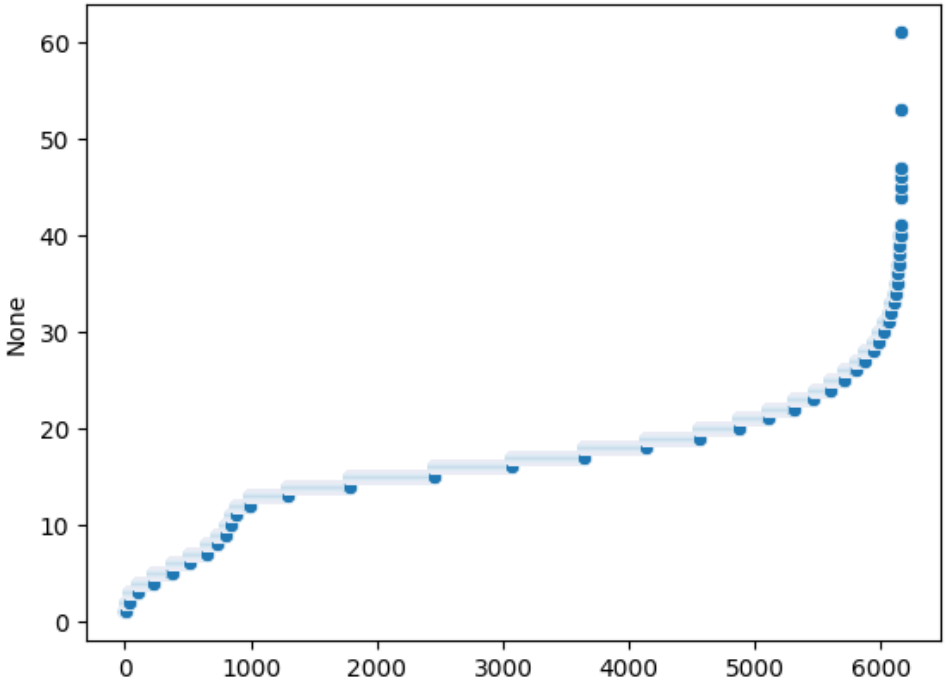

Figure 3: Número de características asignadas a cada ítem.

4. Construcción de la representación TF-IDF

Una vez construida la matriz binaria libro–concepto, se calculó su transformación TF-IDF. En este contexto, el término *tf* corresponde a la presencia o ausencia de un concepto en el libro, mientras que el término *idf* se define como:

$$\mathrm{idf}_i = \log_2\left(\frac{N}{df_i}\right),$$

donde N es el número total de libros y df_i es el número de documentos en los que aparece el concepto i.

La intuición detrás de esta transformación es reducir el peso de los conceptos muy frecuentes y aumentar el peso relativo de los conceptos más específicos. Así, atributos generales como *Novel* o *Fiction* reciben menor peso que atributos raros o altamente discriminativos. El resultado fue una matriz TF-IDF con la misma estructura libro–concepto, pero con pesos reales en lugar de una codificación binaria.

4.1. Mapa de calor de las 300 características más frecuentes

Para inspeccionar la estructura de la matriz TF-IDF se visualizó un mapa de calor usando únicamente las 300 características más frecuentes. Incluso en este subconjunto se observó una matriz altamente dispersa: la mayor parte de las combinaciones libro–característica permanecen en cero, y solo algunas posiciones muestran pesos positivos.

Esto confirma que, aun restringiendo el análisis a las características más comunes, los libros comparten solo una fracción de los conceptos disponibles. Además, como se trata precisamente de conceptos frecuentes, sus pesos TF-IDF tienden a ser relativamente bajos. En consecuencia, el mapa de calor muestra coincidencias parciales entre libros, pero no una superposición masiva de atributos.

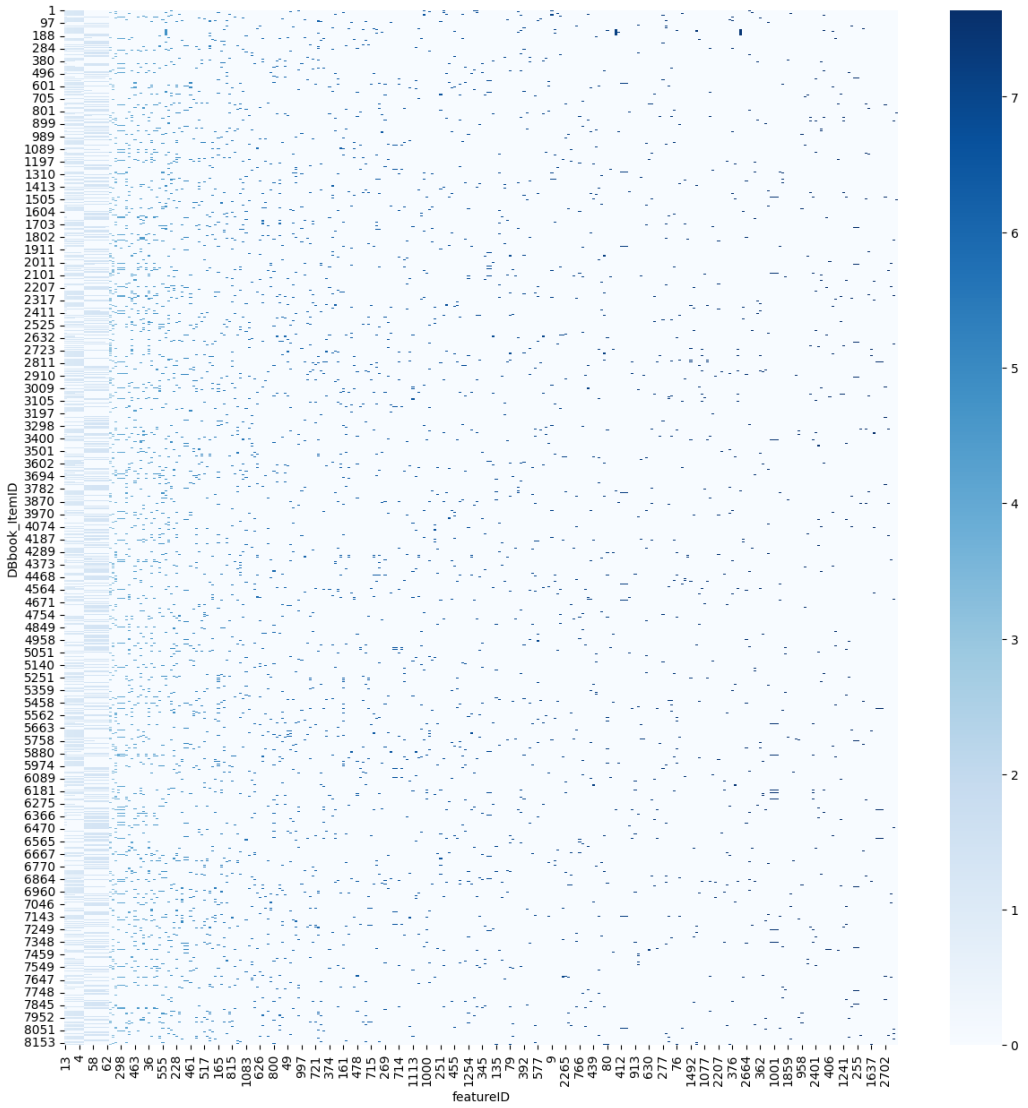

Figure 4: Mapa de calor de la matriz TF-IDF para las 300 características más frecuentes.

4.2. Características más frecuentes

El cruce entre la frecuencia documental y el catálogo de conceptos permitió identificar las características más frecuentes del dataset. Entre ellas destacan conceptos muy generales como `/Abstraction100002137`, `/Communication100033020`, `/WrittenCommunication106349220`, `/Writing106362953`, `/LiteraryComposition106364329`, `/Fiction106367107` y `/Novel106367879`. Esto refuerza la idea de que una parte importante del vocabulario corresponde a conceptos amplios de la ontología, comunes a muchos libros, y por tanto con baja capacidad de discriminación individual.

5. Selección de características

Dado que la representación inicial contenía 12,246 conceptos, se evaluó la conveniencia de conservar todas las columnas de la matriz TF-IDF. El resumen estadístico de las frecuencias mostró una distribución extremadamente sesgada: la mediana fue 1, el percentil 75 fue 3 y el máximo alcanzó 3089. En otras palabras, al menos la mitad de las características aparece en un solo libro y tres cuartas partes aparecen en tres o menos libros.

A partir de este diagnóstico, se concluyó que no era conveniente conservar todo el vocabulario. Muchas características eran demasiado raras, aportaban poca capacidad de generalización y aumentaban tanto la dispersión como el costo computacional del modelo. Por esta razón, se filtraron las columnas con frecuencia menor a 3 y se mantuvieron únicamente las características presentes en al menos tres ítems. Tras este filtrado, la matriz TF-IDF quedó reducida a 3894 columnas, lo que permitió trabajar con una representación más compacta y potencialmente más robusta.

```
# Seleccionamos las columnas cuyo conteo de apariciones es mayor o igual a 3
df_matriz_tf_idf = df_matriz_tf_idf.loc[:, series_suma[series_suma >= 3].index]
df_matriz_tf_idf
```

Listing 1: Filtrado de características por frecuencia en la matriz TF-IDF.

6. Binarización de preferencias y partición train–test

Para convertir el problema en una tarea de clasificación supervisada, se binarizaron los ratings por usuario. La regla utilizada consistió en calcular el rating promedio de cada usuario y etiquetar como `True` aquellas interacciones cuyo rating fuera mayor o igual a su media personal, y como `False` aquellas por debajo de este umbral. Este criterio permite interpretar la variable objetivo como una señal relativa de preferencia, ajustada al comportamiento individual de cada usuario.

- `merge(...)`: Agrega a cada interacción el promedio correspondiente de su usuario (`mean`).
- `df_all_interactions['rate'] >= df_all_interactions['mean']`: Devuelve:
  - **True**: Si el rating es mayor o igual al promedio del usuario.
  - **False**: Si está por debajo.

```
# Combinamos los promedios con las interacciones originales
df_all_interactions = df_all_interactions.merge(df_user_mean, on='DBbook_userID',
    how='left')

# Definimos la clase: 1 (True) si supera el promedio, 0 (False) si no
df_all_interactions['class'] = df_all_interactions['rate'] >= df_all_interactions['
    mean']
df_all_interactions
```

Listing 2: Cálculo de la clase binaria basada en el promedio por usuario.

Posteriormente, el conjunto de interacciones se dividió en entrenamiento y prueba usando una partición 80/20 estratificada por usuario. Con ello se buscó que los usuarios presentes en test también estuvieran representados en train, de forma que el modelo pudiera aprender un perfil por usuario y luego evaluarlo sobre interacciones no vistas del mismo individuo.

7. Selección supervisada de características mediante chi-cuadrado

En filtrado por contenido, el modelo se construyó de forma independiente para cada usuario. Para ello, sobre las interacciones de entrenamiento de un usuario se unió la etiqueta binaria con la representación TF-IDF de los libros y se aplicó la prueba chi-cuadrado, comparando cada característica contra la variable objetivo `class` [2].

El objetivo de este paso fue conservar únicamente aquellas características con capacidad de discriminar entre libros que al usuario le gustan y libros que no le gustan. Las características con peso chi-cuadrado positivo se guardaron en un diccionario indexado por usuario, donde cada llave corresponde al identificador del usuario y cada valor es el conjunto de *features* seleccionadas para él. Debido al costo computacional del procedimiento, esta selección se realizó para los 500 usuarios con mayor número de interacciones en el conjunto de test.

7.1. Análisis de Clasificación: Caso de Estudio Usuario 3852

Para evaluar el desempeño del clasificador KNN ($k = 3$), se analizó el conjunto de entrenamiento del usuario 3852, utilizando las características seleccionadas previamente.Evaluación de PrediccionesA continuación se presenta la comparativa entre la clase real y la predicción obtenida:

7.1.1. Evaluación de predicciones

DBbook_ItemID	Nombre del Libro	Clase Real	Predicción
2578	A Room with a View	True	True
6965	Desert of the Heart	False	False
blue!5 **2839**	**The Bone Doll's Twin**	**False**	**True**
7271	Mr. Midshipman Hornblower	False	False
4760	Geography Club	False	False

Tabla 2: Resultados de predicción para el usuario 3852.

7.1.2. Hipótesis del Error en "The Bone Doll's Twin"

La clasificación falló en el ítem 2839 debido a que el modelo tomó una decisión por mayoría simple en un espacio vectorial donde predominan vecinos positivos. Aunque el vecino más cercano (*Hidden Warrior*) es negativo, los otros dos (*Traitor's Moon* y *Stalking Darkness*) son positivos, sesgando el resultado hacia `True`.

1. **Evidencia por Característica (Feature):** Se analizó la distribución de las características clave para entender la preferencia del usuario:

featureShortname	n_false	n_true	Total	Prop. True
/Fantasy_literature	1	3	4	0.75
/Lynn_Flewelling	1	3	4	0.75
/Category:American_fantasy_novels	2	3	5	0.60
/Category:HarperCollins_books	2	0	2	0.00

Tabla 3: Resumen de evidencia por característica para el usuario 3852.

2. **Sustento de Evidencia en Libros del Usuario:** Se identificaron los libros que aportan peso a estas características y su respectiva clase:

	DBbook_ItemID	class	featureID	featureShortname	name
2	2836	True	92	/Fantasy_literature	Luck in the Shadows
5	2838	True	92	/Fantasy_literature	Traitor's Moon
12	3278	True	92	/Fantasy_literature	Stalking Darkness
8	2837	False	92	/Fantasy_literature	Hidden Warrior
1	6709	False	141	/Category:HarperCollins_books	The Reptile Room
9	2837	False	141	/Category:HarperCollins_books	Hidden Warrior

Tabla 4: Relación de libros y características.

3. **Comparar ítem fallido vs vecinos:** ¿Los vecinos positivos comparten esas features con el ítem fallido?

	DBbook_ItemID	name	featureID	featureShortname
444282	2837	Hidden Warrior	92	/Fantasy_literature
444283	2837	Hidden Warrior	141	/Category:HarperCollins_books
444285	2837	Hidden Warrior	556	/Category:American_fantasy_novels
444286	2837	Hidden Warrior	7650	/Lynn_Flewelling
9270	2838	Traitor's Moon	92	/Fantasy_literature
9271	2838	Traitor's Moon	556	/Category:American_fantasy_novels
9273	2838	Traitor's Moon	7650	/Lynn_Flewelling
9284	2839	The Bone Doll's Twin	92	/Fantasy_literature
9286	2839	The Bone Doll's Twin	141	/Category:HarperCollins_books
9287	2839	The Bone Doll's Twin	556	/Category:American_fantasy_novels
9288	2839	The Bone Doll's Twin	7650	/Lynn_Flewelling
9298	3278	Stalking Darkness	92	/Fantasy_literature
9299	3278	Stalking Darkness	556	/Category:American_fantasy_novels
9301	3278	Stalking Darkness	7650	/Lynn_Flewelling

Tabla 5: Relación de ítems y características.

7.1.3. Conclusión del Análisis

El error se originó porque *The Bone Doll's Twin* comparte múltiples atributos de alta relevancia positiva para este usuario, específicamente /Fantasy_literature y /Lynn_Flewelling (ambas con una proporción de agrado del 75%). Aunque el libro posee la característica /Category:HarperCollins_books (asociada históricamente a ítems negativos para este usuario), el patrón dominante de los vecinos más cercanos en el espacio de fantasía fue suficiente para superar esta señal negativa, resultando en un falso positivo.

8. Modelo KNN sobre representación TF-IDF

Con las características seleccionadas por usuario, se construyó un clasificador `KNeighborsClassifier`. Para cada usuario se tomaron sus libros de entrenamiento, se proyectaron sobre las *features* seleccionadas y se entrenó un modelo binario *le gusta / no le gusta*. Luego, este modelo se aplicó a los ítems del conjunto de test del mismo usuario [2].

En una inspección cualitativa por usuario se observó que algunos errores de clasificación podían explicarse por la presencia de características compartidas con vecinos positivos. Un caso representativo fue *The Bone Doll's Twin*, cuya predicción positiva se debió a que activaba señales como `/Fantasy_literature`, `/Lynn_Flewelling` y `/Category:American_fantasy_novels`, todas ellas más asociadas a interacciones positivas que negativas en el historial del usuario. Aunque también aparecía una señal adversa como `/Category:HarperCollins_books`, la votación mayoritaria de KNN favoreció la clase positiva.

8.1. Ajuste del número de vecinos

Para optimizar el modelo se evaluaron distintos valores de k: $1, 3, 5, 7, 9, 11$. Los resultados mostraron un patrón interesante: valores pequeños de k obtuvieron mejor precisión, mientras que valores más altos mejoraron el recall. El mejor equilibrio entre ambas métricas se alcanzó con $k = 11$, que produjo los mejores resultados globales del experimento TF-IDF:

- precision = 0.5842
- recall = 0.6405
- f1 = 0.6110
- predicciones evaluadas = 2375
- usuarios evaluados = 493

Esto sugiere que un vecindario ligeramente más amplio permite estabilizar la clasificación y reducir la sensibilidad a observaciones individuales, favoreciendo un mejor compromiso entre aciertos positivos y falsos positivos.

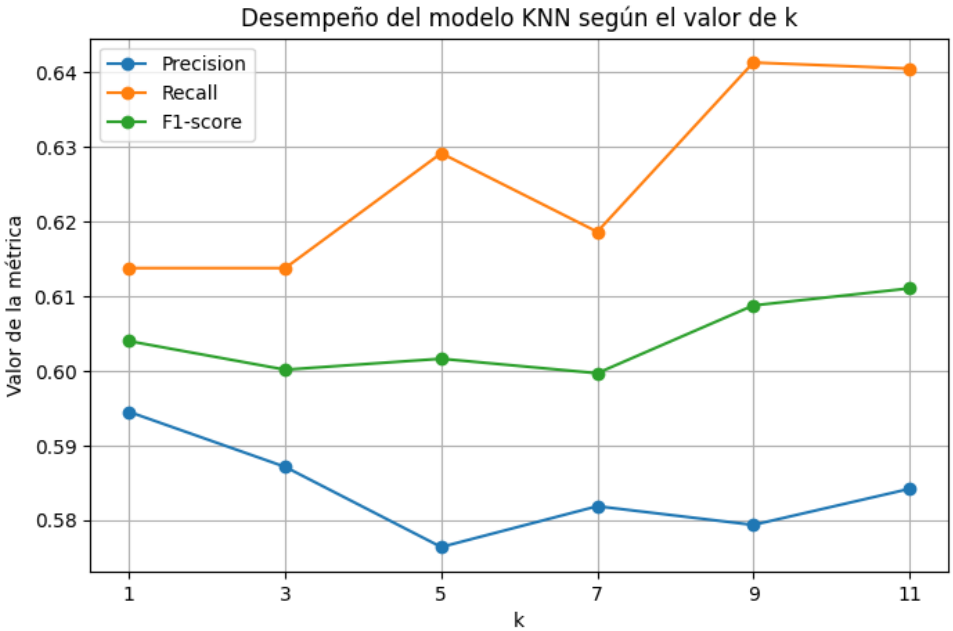

Figure 5: Precision, recall y F1-score del modelo KNN con representación TF-IDF para diferentes valores de k.

9. De TF-IDF a embeddings densos: Word2Vec

La segunda parte del laboratorio abordó una representación alternativa basada en embeddings densos. El punto de partida fue reconocer varias limitaciones de TF-IDF: alta dimensionalidad, fuerte dispersión y ausencia de relaciones semánticas explícitas entre conceptos. En particular, dos libros que traten del mismo tema pero estén descritos con conceptos distintos de DBpedia pueden aparecer como poco similares bajo una comparación puramente basada en coincidencia exacta de *features*.

Para mitigar estas limitaciones, se entrenó un modelo Word2Vec sobre los conceptos de los libros. La idea fue tratar cada libro como una "oración" compuesta por los `featureShortname` asociados al ítem, y cada concepto como una "palabra". De esta forma, conceptos que suelen aparecer juntos dentro de los mismos libros tienden a quedar cercanos en el espacio vectorial [4, 3].

9.1. Entrenamiento del modelo

El modelo Word2Vec se entrenó con una representación de libros como secuencias de conceptos, usando `vector_size=100`, `window=10`, `min_count=2` y arquitectura *Skip-gram*. Esta configuración buscó capturar relaciones semánticas entre conceptos frecuentes sin incluir ruido excesivo de conceptos extremadamente raros [4, 3].

La inspección del vocabulario aprendido y las consultas de similitud mostraron resultados coherentes. Por ejemplo, al consultar los conceptos más similares a `/Fantasy_literature`, el modelo devolvió términos relacionados como `/High_fantasy`, `/Category:Discworld_books`, `/Terry_Pratchett`, `/Margaret_Weis` y otras categorías o autores del mismo campo semántico. Esto sugiere que Word2Vec logró capturar estructura temática más allá de la coincidencia exacta entre conceptos.

9.2. Representación de libros como promedio de embeddings

Una vez entrenado el modelo, cada libro se representó como el promedio de los embeddings de sus conceptos. Así se obtuvo una nueva matriz de representación densa, de tamaño 6164×100, donde cada fila corresponde a un libro y cada columna a una dimensión latente del embedding.

Esta estrategia reduce drásticamente la dimensionalidad respecto a TF-IDF y permite representar libros conceptualmente similares aun cuando no compartan exactamente las mismas características originales. Aunque el promedio de embeddings pierde cierta granularidad y asigna el mismo peso a todos los conceptos del libro, ofrece una representación compacta y semánticamente más rica.

10. Modelo KNN con representación Word2Vec

Utilizando la matriz densa de embeddings por libro, se repitió el proceso de clasificación con KNN. Para un usuario de ejemplo, la representación Word2Vec mostró una tendencia a predecir la clase positiva con mayor frecuencia que la representación TF-IDF. En un caso ilustrativo, el modelo clasificó como positivos todos los ítems del conjunto de prueba del usuario, recuperando correctamente los libros que sí le gustaban, pero cometiendo errores en los ítems negativos.

Este patrón anticipaba una posible mejora en recall a costa de una mayor cantidad de falsos positivos, lo que luego se confirmó en la comparación agregada sobre múltiples usuarios.

11. Comparación entre TF-IDF y Word2Vec

Para comparar ambas estrategias se evaluó el clasificador KNN con $k = 3$ sobre 493 usuarios y 2375 predicciones. Se calcularon métricas promedio por usuario y métricas globales agregadas para cada representación.

Tabla 6: Comparación entre TF-IDF y Word2Vec sobre 493 usuarios evaluados.

Modelo	Prec. prom.	Recall prom.	F1 prom.	Prec. global	Recall global	F1 global
TF-IDF	0.4122	0.5377	0.4314	0.5873	0.6073	0.5971
Word2Vec	0.4947	0.5842	0.4967	0.5737	0.6462	0.6078

Los resultados muestran que **Word2Vec obtuvo mejores métricas promedio por usuario** que TF-IDF en precision, recall y F1-score. En la evaluación global, TF-IDF logró una precision ligeramente superior, pero Word2Vec obtuvo mejor recall y mejor F1. En conjunto, esto indica que la representación densa fue más efectiva para recuperar ítems relevantes, aun a costa de introducir algunos falsos positivos adicionales.

Una explicación plausible es que Word2Vec captura relaciones semánticas entre conceptos relacionados, incluso cuando los libros no comparten exactamente las mismas *features* de DBpedia. En contraste, TF-IDF depende de coincidencias exactas entre columnas, lo que puede limitar la capacidad de generalización cuando dos libros abordan temas similares usando descripciones conceptuales distintas.

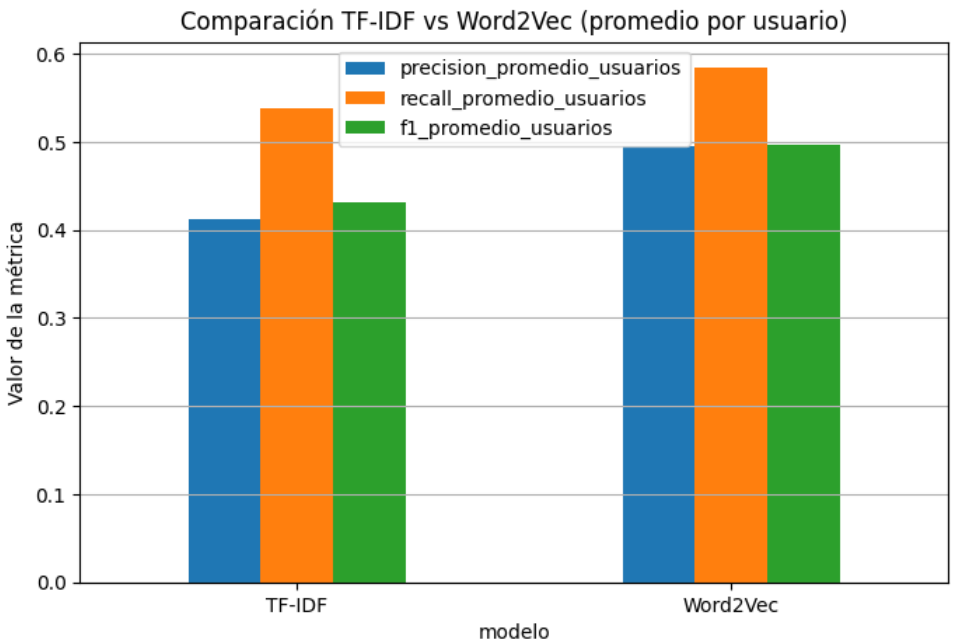

Figure 6: Comparación de precision, recall y F1-score promedio por usuario entre TF-IDF y Word2Vec.

12. Reflexión sobre las representaciones

Desde una perspectiva conceptual, TF-IDF ofrece una representación simple, interpretable y basada en coincidencias exactas, útil cuando interesa saber con precisión qué atributos explican una recomendación. Sin embargo, su alta dimensionalidad y su naturaleza dispersa hacen que el problema sea computacionalmente más costoso y menos robusto frente a variaciones semánticas entre conceptos.

Word2Vec, en cambio, requiere una fase de entrenamiento adicional, pero produce vectores densos y compactos que capturan relaciones de coocurrencia y similitud semántica. Esto lo hace particularmente adecuado cuando dos libros pueden ser conceptualmente cercanos sin compartir exactamente los mismos conceptos de DBpedia.

Estas ideas también se conectan naturalmente con el filtrado colaborativo. Así como en este laboratorio se trató cada libro como una oración de conceptos, en un esquema colaborativo podrían tratarse las secuencias de ítems consumidos por cada usuario como oraciones y entrenar embeddings de ítems. Esta intuición corresponde a enfoques como *Item2Vec*, donde los ítems que aparecen juntos en los historiales de consumo tienden a quedar cerca en el espacio vectorial [5].

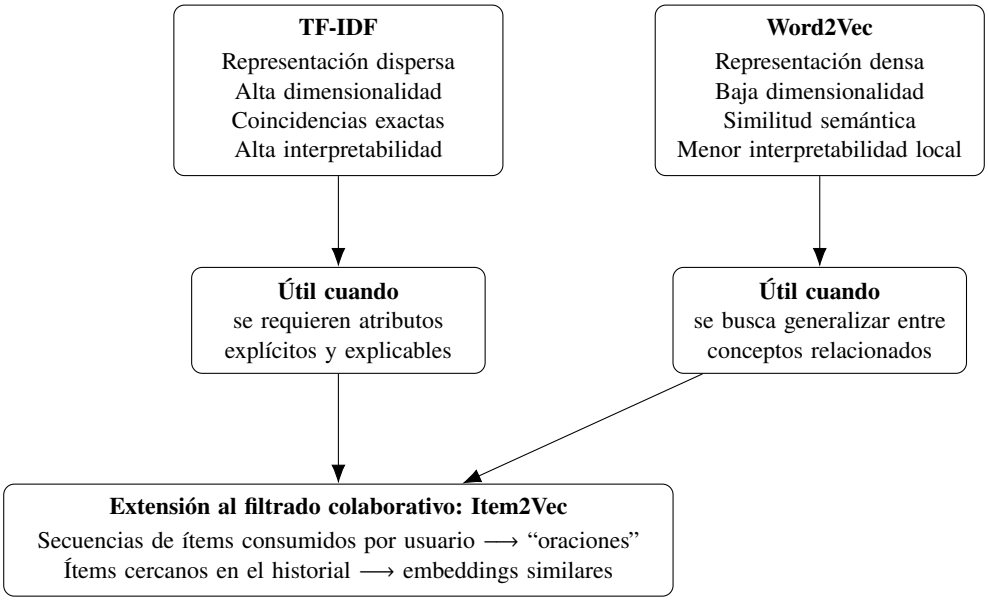

Figure 7: Comparación conceptual entre TF-IDF y Word2Vec, y su extensión hacia esquemas colaborativos tipo Item2Vec.

13. Conclusiones

El laboratorio permitió recorrer un pipeline completo de filtrado por contenido, desde la construcción de una matriz documento–término hasta la evaluación comparativa de representaciones dispersas y densas. En una primera etapa, la representación TF-IDF mostró la fuerte sparsity del problema y la necesidad de aplicar selección de características para reducir dimensionalidad y mejorar capacidad de generalización. Sobre esta base, el modelo KNN por usuario logró un desempeño razonable, optimizado con $k = 11$.

En una segunda etapa, Word2Vec ofreció una representación alternativa más compacta y semánticamente informativa. Los experimentos mostraron que esta estrategia superó a TF-IDF en F1-score y recall, tanto a nivel promedio por usuario como en la evaluación global, lo cual sugiere una mejor capacidad para recuperar ítems relevantes en escenarios donde los conceptos no coinciden exactamente.

En síntesis, TF-IDF sigue siendo una alternativa útil cuando se requiere interpretabilidad y control fino sobre las características, mientras que Word2Vec resulta más apropiado cuando se busca capturar estructura semántica y trabajar con representaciones densas de menor dimensionalidad. La comparación entre ambos enfoques ilustra bien el tránsito desde modelos de representación explícitos hacia embeddings continuos, así como su potencial articulación futura con métodos colaborativos basados en secuencias de ítems.

Referencias

[1] DBpedia Association. *DBpedia Ontology*. Disponible en: https://www.dbpedia.org/resources/ontology/

[2] scikit-learn developers. *scikit-learn: Machine Learning in Python*. Documentación oficial. Disponible en: https://scikit-learn.org/stable/

[3] Gensim Developers. *Word2Vec Model*. Documentación oficial. Disponible en: https://radimrehurek.com/gensim/models/word2vec.html

[4] Tomas Mikolov, Kai Chen, Greg Corrado y Jeffrey Dean. *Efficient Estimation of Word Representations in Vector Space*. arXiv preprint arXiv:1301.3781, 2013. Disponible en: https://arxiv.org/abs/1301.3781

[5] Oren Barkan y Noam Koenigstein. *Item2Vec: Neural Item Embedding for Collaborative Filtering*. arXiv preprint arXiv:1603.04259, 2016. Disponible en: https://arxiv.org/abs/1603.04259